Name: Sebastian Nanghanda

Student No.:225047055

Group: 11

Role : Data and resilience

1. Individual Contribution Statement (Role 4: Data & Resilience Lead)

For Assignment 4, I took on Role 4, which focuses on data management and system resilience. My main job was to figure out how our desktop app, STEMCraft, handles storing data locally on the user's computer and how it syncs that data back to the database when an internet connection becomes available. Because many schools in Namibia have unstable network access, low bandwidth, or expensive mobile data, I had to make sure teachers and equipment custodians can save their work offline without losing anything if the connection drops or the laptop turns off.

I worked on designing the logical data entities, defining access permissions between facilitators and custodians, and fixing database issues in our SQLite schema (`schema_Patched.sql`). I solved five database bugs from our previous A3 cycle (BUG-DB-01 to BUG-DB-05), adding proper table relationships, cascade rules for drafts, JSON validation checks, and automatic status updates. I also wrote ADR 3 to explain our offline sync queue system, which batches changes in sets of up to 20 records at a time.

2. Team Collaboration & Role Dependencies

My work was closely linked with the rest of the team. I received basic requirements and project constraints from upstream roles, and then handed over database structures and status behaviors to the members working on backend development, interface design, testing, and evaluation.

3. Owned Artifacts & Technical Deliverables

Here are the 5 main deliverables I produced for this project:

- **Patched SQLite Database Schema (db/sqlite/schema_Patched.sql):** The updated database file containing tables for session plans, temporary drafts, and the sync queue outbox.
- **Database Triggers and Constraints (schema_Patched.sql):** Triggers that update plan status automatically when sync status changes, plus rules for cascade deletes and foreign keys.
- **Architecture Decision Record 3 (ADR 3):** The architectural write-up covering our decision to use local SQLite storage and a batch sync outbox.
- **Database Verification Test Script (db/sqlite/Test_suite.sql):** A test file containing 10 SQL test cases to make sure foreign keys, JSON validation, and triggers work properly.

- **Logical Data Model & Access Governance Breakdown:** Documentation explaining table relationships, attributes, and user role access rules for facilitators and custodians.

Code Sample: Patched SQLite Schema & Status Trigger

```
-- Session Plans table
CREATE TABLE IF NOT EXISTS session_plans (
   plan_id TEXT PRIMARY KEY,
   user_id TEXT,
   title TEXT NOT NULL,
   target_group TEXT NOT NULL,
   sync_status TEXT DEFAULT 'WAITING_TO_SYNC',
   created_at TEXT NOT NULL DEFAULT (STRFTIME('%Y-%m-%d %H:%M:%f', 'NOW')),
   updated_at TEXT NOT NULL DEFAULT (STRFTIME('%Y-%m-%d %H:%M:%f', 'NOW'))
);

-- Drafts table with JSON check and cascade delete
CREATE TABLE IF NOT EXISTS drafts (
   plan_id TEXT PRIMARY KEY REFERENCES session_plans (plan_id) ON DELETE CASCADE,
   payload_json TEXT NOT NULL CHECK (json_valid(payload_json)),
   updated_at TEXT NOT NULL DEFAULT (STRFTIME('%Y-%m-%d %H:%M:%f', 'NOW'))
);

-- Outbox queue table for tracking offline changes
CREATE TABLE IF NOT EXISTS outbox (
   queue_id INTEGER PRIMARY KEY AUTOINCREMENT,
   plan_id TEXT REFERENCES session_plans (plan_id) ON DELETE SET NULL,
   action TEXT NOT NULL,
   status TEXT NOT NULL DEFAULT 'WAITING_TO_SYNC' CHECK (status IN ('WAITING_TO_SYNC', 'SYNCING', 'SYNCED', 'FAILED')),
   attempt_count INTEGER NOT NULL DEFAULT 0 CHECK (attempt_count >= 0),
   created_at TEXT NOT NULL DEFAULT (STRFTIME('%Y-%m-%d %H:%M:%f', 'NOW'))
);
CREATE INDEX IF NOT EXISTS idx_outbox_status ON outbox (status);

-- Trigger that keeps session_plans sync_status in sync with outbox table
CREATE TRIGGER IF NOT EXISTS trg_sync_outbox_to_plan_update
AFTER UPDATE OF status ON outbox
BEGIN
   UPDATE session_plans
   SET sync_status = NEW.status,
      updated_at = STRFTIME('%Y-%m-%d %H:%M:%f', 'NOW')
   WHERE plan_id = NEW.plan_id;
END;
```

4. Requirement-to-Test Traceability Matrix

Req ID & Priority	Requirement Summary	My Database / Architecture Implementation	Validation Check & Outcome

FR5 (Must)	Plans made offline queue up and sync when internet comes back.	outbox queue table and batching setup (up to 20 items per batch).	DB-TS-06 & TC-SYN-01: Status changes correctly from WAITING_TO_SYNC to SYNCED (Pass).
FR6 (Must)	Plans in progress autosave without losing work.	drafts table with json_valid check on saving data.	DB-TS-05 & T3: App restores saved draft after restarting (Pass).
FR7 (Should)	Custodian tracks equipment checkouts, dates, and return status.	equipment_kits table and separate view for equipment custodian.	Manual SQL check: Checkout records stay intact after application restart (Pass).
QR1 (Must)	Local work is kept safe until server confirms upload.	Local storage first; outbox entries remain until server confirmation.	DB-TS-04: Plan deletion retains historical log record (Pass).
QR2 (Must)	Sync uses small batches without downloading heavy files.	Index idx_outbox_status added to outbox table for fast lookups.	DB-TS-10: Verified outbox status queries use table index (Pass).

5. Failure Reflection & Lessons Learned

One issue I ran into while setting up the database was linking the drafts table directly to the session plans table using a foreign key constraint. I wanted to make sure that deleting a plan would clean up its draft, so I used an ON DELETE CASCADE rule. However, requirement FR6 says that a draft should start saving automatically as soon as a teacher starts typing—even before they click save or finalize.

Because of that rule, the initial test failed when saving a brand new draft because the parent plan ID didn't exist in the database yet. To fix this, I worked with Sebastian (Role 5) to make sure the app creates a basic placeholder row in the main table first whenever a user starts a new plan. That way, autosave can write drafts smoothly without hitting a database error.

Another point to consider is local storage on shared school computers. Since we save everything locally first and don't require user logins, old synced plans just sit on the hard drive. For future updates, adding an automatic cleanup task to clear out old synced records after the server confirms them would prevent the computer from running out of disk space over time.